

GraminCare

Complete Feature Specification — For Team Reference

Problem Statement ID	26133
Organization	Government of Maharashtra
Department	Maharashtra State Innovation Society
Category / Theme	Software — MedTech / HealthTech
Document Purpose	Full feature breakdown so every team member understands what to build, why, and how it fits together

Table of Contents

- 1. How to Read This Document
- 2. User Roles / Actors in the System
- 3. Feature 1 — Patient Registration & Offline Data Capture
- 4. Feature 2 — Digital Triage Engine
- 5. Feature 3 — Assisted Teleconsultation
- 6. Feature 4 — Appointment & Queue Management
- 7. Feature 5 — Longitudinal Patient Records
- 8. Feature 6 — Referral Tracking
- 9. Feature 7 — Medicine & Diagnostic Availability
- 10. Feature 8 — High-Risk Patient Follow-Up
- 11. Feature 9 — Facility & District Dashboards
- 12. Feature 10 — Offline-First Sync & Connectivity Layer
- 13. Feature 11 — Multilingual & Voice Interaction
- 14. Feature 12 — Emergency Escalation (SOS)
- 15. Feature 13 — Interoperability (FHIR / ABDM)
- 16. Feature 14 — AI Patient Assistant (Ollama Local Agent)
- 17. Data Model Summary
- 18. Team Split Suggestion

1. How to Read This Document

Every feature below follows the same structure so anyone on the team can jump to their assigned module and know exactly what to build:

- **What it is** — one-line plain description
- **Why it matters** — which real-world pain point it solves
- **Who uses it** — the actor(s) interacting with this feature
- **How it works (flow)** — step-by-step user flow
- **Data involved** — what gets stored/read
- **Screens/UI needed** — concrete screens to design/build
- **Build priority** — **MUST HAVE** (Phase 1, build this first), **SHOULD HAVE** (Phase 2), **NICE TO HAVE** (Phase 3, mention in pitch only)
- **Edge cases** — things that will break the demo if ignored

2. User Roles / Actors in the System

Actor	Description	Main Device
ASHA / ANM Worker	Frontline health worker visiting villages; captures patient data, assists teleconsultation	Android tablet/phone app (offline-first)
PHC Doctor	Doctor at Primary Health Centre; conducts teleconsultation, reviews triage, creates referrals	Web/desktop dashboard
District Hospital Staff	Receives referrals, manages specialist consults	Web dashboard
Patient / Family	Receives care, appointment reminders, follow-up calls	Feature phone (SMS/IVR) or smartphone
Facility Admin / Pharmacist	Updates medicine and diagnostic stock	Web/tablet app
Block/District Health Officer	Monitors quality, load, and outcomes across facilities	Web dashboard (analytics)

1 Feature 1 — Patient Registration & Offline Data Capture

MUST HAVE

What it is

The entry point of the whole system — a health worker registers a patient and records vitals/symptoms, even with zero internet connectivity.

Why it matters

Most villages this targets have no signal. If data capture requires internet, the entire product fails at step one.

Who uses it

ASHA/ANM worker (primary), patient (subject)

How it works (flow)

1. Worker opens app, taps "New Patient" or searches existing patient by name/phone/ID
2. Enters/updates: name, age, gender, village, phone (if any), Aadhaar/ABHA ID (optional)
3. Records vitals: BP, temperature, pulse, weight, SpO2 (manual entry or Bluetooth device if available)
4. Notes symptoms via structured checklist + free text/voice note
5. Optionally attaches a photo (e.g., wound, rash)
6. Data is saved to local device storage instantly — no internet required
7. Record is queued for sync (see Feature 10)

Data involved

`Patient` (id, name, age, gender, village, phone, ABHA_id), `Visit` (patient_id, worker_id, timestamp, vitals, symptoms, photos, gps_location)

Screens/UI needed

- Patient search/list screen
- New patient registration form
- Vitals entry form (numeric inputs with normal-range hints)
- Symptom checklist + voice/text notes
- Sync status indicator (pending/synced icon)

Edge cases

- Duplicate patient entries (same person registered twice) — add fuzzy name+village matching before creating new record
- App killed mid-entry — autosave draft locally every few seconds
- No phone number available — allow registration without it, flag for follow-up via village kiosk instead

2 Feature 2 — Digital Triage Engine

MUST HAVE

What it is

A rule-based scoring system that classifies each patient visit as **Red (urgent)**, **Yellow (needs consult soon)**, or **Green (routine)** based on vitals and symptoms.

Why it matters

Health workers aren't clinicians. Automated triage tells them who needs immediate escalation vs. who can wait for the next teleconsultation slot — directly reduces delayed care.

Who uses it

ASHA worker (sees the result), PHC doctor (reviews the reasoning)

How it works (flow)

1. Runs automatically right after vitals + symptoms are entered (Feature 1)
2. Applies rule set, e.g.: BP > 180/120 → Red; fever + convulsion in child → Red; mild cough, normal vitals → Green
3. Shows color-coded result with the specific reason ("Flagged Red: BP 190/125")
4. If Red → prompts immediate teleconsultation or emergency escalation (Feature 12)
5. If Yellow/Green → adds to consultation queue at appropriate priority

Data involved

`TriageRule` (condition, threshold, resulting_level), `Visit.triage_result` (level, matched_rules[])

Screens/UI needed

- Triage result banner (red/yellow/green) shown right after vitals entry
- "Why this result" expandable explanation

Edge cases

- Missing vitals (e.g., no BP machine available) — triage should still run on available data and clearly show "incomplete data" rather than silently skipping
- Conflicting rules matching — always escalate to the higher urgency level

Note: keep this rule-based for the hackathon — explain in the pitch that Phase 3 could add ML-based triage with explainability.

3 Feature 3 — Assisted Teleconsultation

MUST HAVE

What it is

Video/audio call between the patient (with the health worker physically present to assist) and a doctor at the PHC.

Why it matters

Replaces the need for a patient to physically travel to a PHC/hospital for a first consult — the single biggest access barrier described in the problem statement.

Who uses it

ASHA worker (facilitates), patient, PHC doctor

How it works (flow)

1. Worker taps "Start Consultation" from a triaged patient record
2. Request appears in doctor's queue (prioritized by triage level)
3. Doctor accepts → video/audio call starts (WebRTC), with the patient's vitals/history visible side-by-side to the doctor
4. Worker relays physical findings the doctor asks for (e.g., "check for swelling")
5. Doctor enters diagnosis notes + prescription during/after the call
6. Call ends → consultation summary saved to patient record

Data involved

Consultation (visit_id, doctor_id, start_time, end_time, call_type, diagnosis_notes, prescription)

Screens/UI needed

- Doctor's incoming consultation queue (sorted by triage urgency)
- Video call screen with patient info panel alongside
- Post-call notes/prescription entry form

Edge cases

- Bandwidth too low for video — auto-fallback to audio-only call
- Call drops mid-consult — save partial notes, allow doctor to resume/finish async
- No connectivity at all — queue consultation request, notify worker when a sync window is available (Feature 10)

4 Feature 4 — Appointment & Queue Management

MUST HAVE

What it is

Lets patients/workers book a teleconsultation or in-person slot, and gives facilities a live queue view.

Why it matters

Reduces wasted travel/waiting — patients know when to actually show up instead of waiting all day at a facility.

Who uses it

ASHA worker (books on behalf of patient), patient (via SMS/IVR), PHC staff (manages queue)

How it works (flow)

1. Worker or patient requests a slot (in-app, or via SMS keyword/IVR call for feature-phone users)
2. System shows available slots at nearest PHC, filtered by triage urgency
3. Booking confirmed → SMS/IVR confirmation sent to patient
4. Facility dashboard shows live queue for the day, updated as patients check in
5. Reminder sent 1 day and 1 hour before appointment (where phone available)

Data involved

Appointment (patient_id, facility_id, slot_time, status: booked/checked-in/completed/no-show)

Screens/UI needed

- Slot picker (calendar/list view)
- Facility-side queue board (today's appointments, live status)

Edge cases

- Double-booking same slot — lock slot on booking attempt
- No-shows — auto-release slot after grace period, log for follow-up

5 Feature 5 — Longitudinal Patient Records

MUST HAVE

What it is

A single patient record that follows the person across every facility they visit — sub-centre, PHC, rural hospital, district hospital — instead of each facility keeping its own paper file.

Why it matters

This is the "fragmented medical records" problem named directly in the problem statement. Without this, every referral starts from zero.

Who uses it

Every clinical actor (worker, doctor, hospital staff)

How it works (flow)

1. Patient gets one unique ID at first registration (linkable to ABHA if available)
2. Every visit, consultation, referral, prescription, and follow-up attaches to that one ID
3. Any authorized facility can pull up the full timeline instantly (once synced)
4. Timeline view shows chronological history: visits, diagnoses, medicines given, referrals

Data involved

Patient (central record), linked Visit , Consultation , Referral , Prescription records — all keyed by patient_id

Screens/UI needed

- Patient timeline view (chronological card list)
- Search by patient ID/name/phone across facilities

Edge cases

- Same patient registered at two facilities with slightly different name spelling — needs a merge/dedupe mechanism
- Privacy — only authorized facility staff should see full history; log access

6 Feature 6 — Referral Tracking

MUST HAVE

What it is

Tracks a patient referred from one level of care to the next (e.g., PHC → district hospital), and flags if they never actually reached the referred facility.

Why it matters

"Delayed referrals" and incomplete follow-through are named explicitly in the problem statement as a key gap.

Who uses it

PHC doctor (creates referral), district hospital staff (receives), health officer (monitors completion rate)

How it works (flow)

1. Doctor creates a referral during/after consultation, selecting destination facility + urgency + reason
2. Referral appears in destination facility's incoming queue
3. Status updates: Created → Patient En Route → Arrived → Seen → Closed
4. If patient doesn't arrive within expected window, system flags "referral at risk" for follow-up call

Data involved

Referral (patient_id, from_facility, to_facility, reason, urgency, status, created_at, resolved_at)

Screens/UI needed

- Referral creation form
- Incoming referral queue (destination facility side)
- Referral status tracker/timeline

Edge cases

- Patient goes to a different facility than referred — allow manual status update/correction
- Referral never actioned — auto-escalate to health officer after X hours

7 Feature 7 — Medicine & Diagnostic Availability

SHOULD HAVE

What it is

Real-time (or near-real-time) visibility into what medicines and diagnostic tests are actually available at a given PHC, so patients aren't sent on a wasted trip.

Why it matters

Directly named in expected outcomes: "improved medicine/diagnostic availability visibility."

Who uses it

Pharmacist/facility admin (updates stock), doctor (checks before referring/prescribing), worker (checks before sending patient)

How it works (flow)

- 1. Facility admin updates stock levels (simple in/out counts) periodically, or marks items as available/low/out
- 2. Doctor or worker checks stock status for a specific facility before prescribing or referring
- 3. Low-stock items are highlighted; out-of-stock items suggest nearest facility that has it

Data involved

Facility, InventoryItem (facility_id, item_name, category: medicine/diagnostic, status, last_updated)

Screens/UI needed

- Stock update form (admin side)
- Stock lookup screen (searchable by item + facility)

Edge cases

- Stale data (admin forgot to update) — show "last updated X days ago" so users can judge reliability

8 Feature 8 — High-Risk Patient Follow-Up

SHOULD HAVE

What it is

Automated reminders and recall lists for patients needing ongoing tracking — pregnant women (ANC visits), children (immunization schedule), and chronic disease patients (diabetes/hypertension check-ins).

Why it matters

Named directly as an expected outcome: "better follow-up for maternal, child and chronic conditions."

Who uses it

ASHA worker (executes follow-up visits), system (auto-generates reminders)

How it works (flow)

- 1. Patient tagged as "high-risk" category at registration/consultation (pregnancy, chronic condition, infant)
- 2. System auto-schedules the next required check-in based on standard protocol (e.g., ANC visit intervals)
- 3. Worker's app shows a daily "due today/overdue" follow-up list for their assigned village(s)
- 4. SMS reminder sent to patient's phone (if available) alongside worker visit

Data involved

FollowUpSchedule (patient_id, category, due_date, status: pending/done/missed)

Screens/UI needed

- Worker's daily follow-up task list
- High-risk patient flag/badge on patient profile

Edge cases

- Worker on leave/reassigned — follow-up list should be reassignable, not tied permanently to one worker

9 Feature 9 — Facility & District Dashboards

MUST HAVE

What it is

Dashboards for facility staff and health officers to see load, referrals, stock, and quality metrics at a glance.

Why it matters

Supports "accountability" and "quality monitoring" — explicitly named outcomes — and gives judges a strong visual to see system-wide impact, not just one patient's journey.

Who uses it

PHC staff, district/block health officers

How it works (flow)

- 1. Facility-level view: today's queue, pending referrals, stock alerts, consultations completed
- 2. District-level view: aggregated stats across facilities — consultation volume, average wait time, referral completion rate, high-risk follow-up compliance
- 3. Drill-down from district view into a specific facility or patient case

Data involved

Aggregated queries across `Visit` , `Consultation` , `Referral` , `InventoryItem` , `FollowUpSchedule`

Screens/UI needed

- Facility dashboard (cards/counters + queue list)
- District dashboard (charts: bar/line for volume and completion rates)

Edge cases

- Dashboard should degrade gracefully with small/demo data — don't let empty charts look broken

10 Feature 10 — Offline-First Sync & Connectivity Layer

MUST HAVE

What it is

The underlying mechanism that lets everything above work with zero or intermittent connectivity, and the physical/network layer design for mountain villages with no roads.

Why it matters

This is your core differentiator vs. generic telemedicine apps — directly answers "low-connectivity environments" in the problem statement.

Who uses it

Invisible to end users — but the worker sees sync status indicators

How it works (flow)

- 1. All writes (registration, vitals, triage) go to local device storage first, instantly, regardless of connectivity
- 2. A background sync process attempts to push queued changes whenever connectivity is detected
- 3. Conflict resolution: last-write-wins for most fields; flag conflicts on critical fields (e.g., diagnosis) for manual review
- 4. Network layer options for the village-to-PHC hop (present as architecture, not necessarily built):
 - **LoRa mesh** — low-power, long-range (2-15 km), good for small data packets (vitals, alerts), cheap hardware
 - **Point-to-point WiFi** — directional antennas, line-of-sight, higher bandwidth (can carry teleconsultation), precedent: AirJaldi-style deployments in Himachal
 - **Satellite backhaul** — no line-of-sight needed, higher cost, positioned as future-scale option
 - **Village kiosk/hub** — solar-powered device that batches data for a whole village and syncs when it reaches a connectivity point, so not every phone needs its own signal

Data involved

`SyncQueue` (record_type, record_id, payload, status: pending/synced/conflict, created_at)

Screens/UI needed

- Sync status icon on every record (pending/synced)
- "Force sync" button for when connectivity is manually detected

Edge cases

- App used on multiple devices for same patient — needs conflict handling, not just overwrite
- Very large photo attachments clog sync queue on slow links — compress images before queuing

Demo tip: simulate this live by toggling device WiFi off, registering a patient, then turning WiFi on and showing the sync happen — very convincing for judges.

11 Feature 11 — Multilingual & Voice Interaction

SHOULD HAVE

What it is

Lets patients and workers interact in their local language, by voice where literacy is a barrier.

Why it matters

Named directly: "multilingual interaction" and "health literacy" barriers in the problem statement.

Who uses it

Patient, ASHA worker

How it works (flow)

- 1. UI text available in Marathi/Hindi/English toggle at minimum
- 2. Symptom notes can be recorded as voice notes and optionally transcribed (speech-to-text) using Bhashini or Google STT
- 3. SMS/IVR reminders sent in the patient's chosen language

Data involved

User.preferred_language , voice note attachments on Visit

Screens/UI needed

- Language selector (onboarding + settings)
- Voice note record button on symptom entry screen

Edge cases

- Transcription API unavailable offline — store raw audio, transcribe later on sync

12 Feature 12 — Emergency Escalation (SOS)

SHOULD HAVE

What it is

A one-tap emergency alert from the worker's app that immediately notifies the nearest facility/ambulance with patient vitals and GPS location.

Why it matters

For Red-triage cases in remote areas, minutes matter — this shortcuts the normal queue/booking flow entirely.

Who uses it

ASHA worker

How it works (flow)

- 1. Worker taps SOS button from a patient's triage screen (auto-suggested if triage = Red)
- 2. Sends alert with patient snapshot (vitals, symptoms, GPS) via any available channel — app push if online, SMS fallback if not
- 3. Nearest facility/ambulance dispatcher sees alert prioritized above normal queue

Data involved

EmergencyAlert (patient_id, gps_location, vitals_snapshot, status, timestamp)

Screens/UI needed

- SOS button (prominent, hard to miss on Red-triage screens)
- Dispatcher-side incoming alert view

Edge cases

- No connectivity at all — fall back to a pre-set emergency SMS/USSD code if even SMS can't get through

13 Feature 13 — Interoperability (FHIR / ABDM)

NICE TO HAVE

What it is

Structuring patient records so they can, in principle, be exchanged with other health systems using India's ABDM (Ayushman Bharat Digital Mission) and HL7 FHIR standards.

Why it matters

Named directly: "interoperable health records based on approved standards." Judges will value this even as a design decision, not a fully built integration.

Who uses it

System-to-system (not user-facing directly)

How it works (flow)

- 1. Design core data model (Patient, Visit, Observation) to map cleanly onto FHIR resource types
- 2. For the hackathon: implement a basic FHIR-format export endpoint for a patient record (not full ABDM certification)

Build guidance

Don't attempt real ABDM registration/certification within a hackathon — that's a multi-month government process. Instead, say in the pitch: "designed to align with FHIR resource structures for future ABDM integration," and show one exported FHIR-JSON sample as proof of concept.

SHOULD HAVE

What it is

A conversational AI agent, powered by **Ollama running locally** (not a cloud API), that patients or workers can chat with for symptom triage-assist, appointment booking, medicine stock checks, and follow-up reminders — entirely offline.

Why it matters

Directly addresses "limited awareness of available services" and "health literacy" from the problem statement. Running it on Ollama (local, on-device) instead of a cloud API means the agent works with **zero internet** — reinforcing the project's core offline-first story instead of contradicting it. It also means no API cost and no dependency on venue WiFi during the demo.

Who uses it

Patient (primary), ASHA worker (assists patients who can't type/read)

Why Ollama specifically (vs. a cloud LLM)

Factor	Ollama (local)	Cloud API (e.g. Gemini)
Cost	Free, forever	Free tier, rate-limited
Works with no internet	Yes — genuinely offline	No
Fits project's offline-first narrative	Yes, directly reinforces it	Contradicts it
Multilingual quality (Hindi/Marathi)	Weaker on small models	Stronger
Demo reliability	Runs on your own laptop, no venue WiFi risk	Depends on venue internet

How it works (flow)

- Ollama runs locally on the demo laptop/server (localhost:11434), no internet required
- Patient/worker opens chat screen, types or speaks a message ("I have fever since 2 days", "when is my next checkup?")
- App sends the message + conversation history to the local Ollama server, along with a strict system prompt (see below)
- Agent replies in plain language and tags its own intent: symptom_check , appointment , stock_check , follow_up , escalate , or general
- App reads the intent tag and routes accordingly:
 - escalate → triggers urgent consult/SOS flow (Feature 3 / Feature 12) immediately
 - appointment → opens booking flow (Feature 4)
 - stock_check → looks up medicine availability (Feature 7)
 - follow_up → checks their follow-up schedule (Feature 8)
- Full conversation transcript saves to the patient's record so the doctor has context before the actual consult

Critical guardrail — the system prompt

This is the single most important part to get right. The agent must never diagnose or prescribe — only triage-assist and route to a human when unsure. Use this system prompt (or a close variant):

```
You are a health triage-assist agent for GraminCare, serving rural patients in Maharashtra.

RULES YOU MUST FOLLOW:
1. NEVER diagnose a condition or name a disease.
2. NEVER prescribe or recommend medicines or dosages.
3. If symptoms sound urgent or you are unsure, respond with escalation and set intent to "escalate".
4. For routine questions (appointments, medicine stock, follow-up dates), help directly.
5. Always respond in the patient's language (Marathi/Hindi/English).
6. Keep responses short (2-3 sentences) and simple - assume low health literacy.
7. At the end of every response, output a JSON tag on a new line:
{"intent": "symptom_check" | "appointment" | "stock_check" | "follow_up" | "escalate" | "general"}
```

Setup steps (for whoever builds this)

- Install Ollama: curl -fsSL https://ollama.com/install.sh | sh (Mac/Linux) or download installer (Windows)
- Pull a lightweight model: ollama pull llama3.2:3b (fine for a laptop demo, no GPU needed)
- Verify it runs: ollama run llama3.2:3b "Hello" — Ollama auto-starts a local server at http://localhost:11434
- Call it from the app backend:

```
async function chatWithAgent(patientMessage, history) {
  const response = await fetch("http://localhost:11434/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "llama3.2:3b",
      messages: [
        { role: "system", content: SYSTEM_PROMPT },
        ...history,
        { role: "user", content: patientMessage }
      ],
      stream: false
    })
  });
};
```

```
const data = await response.json();
const text = data.message.content;

const intentMatch = text.match(/\{"intent":\s*"(\w+)"\}/);
const intent = intentMatch ? intentMatch[1] : "general";

return {
  reply: text.replace(/\{"intent".*\}/, "").trim(),
  intent
};
}
```

Data involved

AgentConversation (patient_id, timestamp, transcript, detected_intent, escalated: true/false)

Screens/UI needed

- Chat interface (text + optional mic button using Web Speech API for voice-to-text) in the patient-facing app
- "Escalated conversations" view for the doctor/worker to review before their consult

Edge cases

- Small local model gives a weak/wrong-language reply — test Hindi/Marathi quality early; if too weak, keep the live demo in English and mention Bhashini/cloud LLM as the Phase 2 production upgrade for multilingual quality
- Model runs slowly on an underpowered laptop — use the smallest model (3b) for the demo machine, and pre-warm the model (run one query before going on stage) so first response isn't slow
- Agent doesn't reliably output the intent JSON tag — add a fallback keyword scan (e.g., words like "pain", "emergency", "can't breathe") as a second safety net for escalation

Test this before demo day (non-negotiable)

Confirm urgent-sounding messages ("severe chest pain", "baby not breathing") reliably trigger the escalation flow every single time — this is the safety guardrail judges are most likely to probe directly.

15. Data Model Summary

Entity	Key Fields	Used By Features
Patient	id, name, age, gender, village, phone, ABHA_id, preferred_language	1, 5, 11
Visit	patient_id, worker_id, timestamp, vitals, symptoms, photos, gps, triage_result	1, 2, 5
Consultation	visit_id, doctor_id, start/end time, notes, prescription	3, 5
Appointment	patient_id, facility_id, slot_time, status	4
Referral	patient_id, from_facility, to_facility, reason, urgency, status	6, 5
InventoryItem	facility_id, item_name, category, status, last_updated	7
FollowUpSchedule	patient_id, category, due_date, status	8
EmergencyAlert	patient_id, gps_location, vitals_snapshot, status	12
SyncQueue	record_type, record_id, payload, status	10
Facility	id, name, type (sub-centre/PHC/rural/district), location	All
AgentConversation	patient_id, timestamp, transcript, detected_intent, escalated	14

16. Team Split Suggestion

Track	Owns Features	Suggested Skillset
Frontend – Field App	1 (Registration), 2 (Triage UI), 8 (Follow-up list), 11 (Language/voice)	Flutter/React Native + offline storage
Frontend – Facility/Web	3 (Doctor console), 4 (Queue), 9 (Dashboards)	React + charting
Backend & Data	5 (Records), 6 (Referrals), 7 (Inventory), 13 (FHIR export)	Node.js/Django + PostgreSQL
Connectivity/Infra	10 (Sync engine), 12 (SOS), architecture diagram	Systems/networking-minded member
AI Agent	14 (Ollama patient assistant), guardrail prompt design	Backend member comfortable with local LLM setup
Pitch & Design	Demo script, slides, UI polish	Anyone strong at storytelling/design